XGBoost

Libraries Used

- pandas
- numpy
- sklearn.preprocessing (LabelEncoder)
- sklearn.model_selection (train_test_split, StratifiedKFold, RandomizedSearchCV, make_scorer)
- sklearn.metrics (f1_score, accuracy_score, recall_score, precision_score, roc_auc_score, classification_report, confusion_matrix, roc_curve, precision_recall_curve, average_precision_score)
- imblearn.over_sampling (SMOTE)
- xgboost (XGBClassifier)
- matplotlib.pyplot
- seaborn
- joblib

Data Preprocessing

1. Handling Incorrect Data Types

TotalCharges was converted to a numeric field named TotalCharges_numeric using pd.to_numeric(). After conversion, missing values were checked. The mean difference between the actual TotalCharges and the estimated value (MonthlyCharges * tenure) was calculated across all non-missing rows and used to impute the missing entries:

TotalCharges = (MonthlyCharges * tenure) + Mean Difference

The original TotalCharges column and the helper columns (calc_total, diff) were dropped after imputation.

2. Removing Irrelevant Columns

The column customerID was removed as it acts only as a row identifier and carries no predictive value.

3. Encoding Categorical Features

All categorical columns (excluding the target variable Churn) were identified using select_dtypes and encoded using LabelEncoder, converting each column to integer representation. The target variable Churn was encoded separately by mapping 'No' to 0 and 'Yes' to 1 after stripping whitespace.

4. Data Splitting

The data was split into training and testing sets (80:20) using stratified splitting to preserve the original class distribution across both sets.

5. Handling Class Imbalance

SMOTE (Synthetic Minority Oversampling Technique) was applied exclusively to the training data to address class imbalance. Applying SMOTE only to training data prevents data leakage into the test set, ensuring that evaluation metrics reflect real-world generalization.

- X_train shape after SMOTE: (8278, 19)
- X_test shape: (1409, 19)
- y_train distribution after SMOTE: 50% Churn, 50% Non-Churn
- y_test distribution: 73.5% Non-Churn, 26.5% Churn

Model Implementation

XGBoost (Extreme Gradient Boosting) is a highly efficient and scalable implementation of the gradient boosting framework. It builds an ensemble of decision trees sequentially, where each tree corrects the errors made by the previous one. XGBoost incorporates regularization (L1 and L2) natively, making it robust against overfitting while achieving high predictive performance.

Hyperparameter Tuning Using RandomizedSearchCV

RandomizedSearchCV was used to explore the hyperparameter space efficiently by sampling 25 random combinations (n_iter=25) from the defined parameter distributions, rather than exhaustively testing all combinations.

Parameter Search Space:

Hyperparameter	Search Space
n_estimators	100, 150
max_depth	3, 5, 7
learning_rate	0.05, 0.1
subsample	0.8, 1
colsample_bytree	0.8, 1

gamma	0, 0.1
min_child_weight	1, 3

Methodology:

- Stratified 3-Fold Cross-Validation (StratifiedKFold, n_splits=3) was applied to maintain class distribution in each fold.
- F1-score was used as the scoring metric to optimize for churn detection under class imbalance.
- The XGBClassifier was initialized with use_label_encoder=False and eval_metric='logloss'.
- The best parameter combination was identified and used to train the final model.

Model Training

The final XGBoost model was trained using the best parameters found by RandomizedSearchCV on the SMOTE-resampled training data (X_train_res, y_train_res). The model was initialized with use_label_encoder=False and eval_metric='logloss' to suppress deprecation warnings and use log loss as the internal evaluation metric during training.

Threshold Tuning

The default classification threshold of 0.5 may not be optimal for imbalanced datasets. To address this, the threshold was systematically tuned:

- The model's predicted probabilities on the test set were evaluated across thresholds ranging from 0.1 to 0.7 in steps of 0.01.
- At each threshold, the F1-score was computed on the test set.
- The threshold yielding the highest F1-score was selected as the final threshold.

Final Best-Chosen Threshold: 0.40

A lower threshold than the default 0.5 aligns with the business priority of maximizing recall for the churn class, accepting more false positives in exchange for capturing more true churners. Missing a churner is costlier than incorrectly flagging a loyal customer.

Model Evaluation

1. Evaluation Metrics (Train vs Test)

	Accuracy	Precision	Recall	F1 Score	AUC Score
Train Set	0.9052	0.8660	0.9587	0.9100	0.9729
Test Set	0.7700	0.5517	0.7139	0.6224	0.8322

Interpretation:

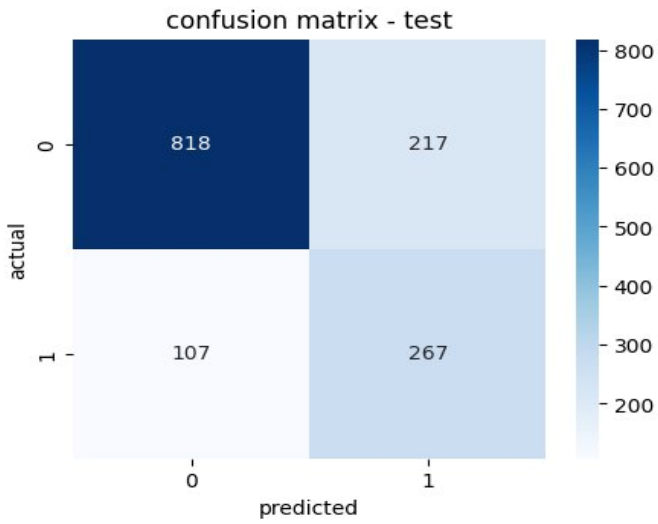
- Train accuracy (0.9052) and AUC (0.9729) are notably higher than test accuracy (0.7700) and AUC (0.8322), indicating mild overfitting — expected with boosting algorithms trained on SMOTE-augmented data.
- Test recall (0.7139) is the most critical metric for this task. Although slightly lower than the train recall (0.9587), it still captures a substantial proportion of actual churners.
- Test precision (0.5517) is lower than training precision (0.8660), which is typical when SMOTE is applied only to training data, as the oversampled distribution does not reflect the real test distribution, leading to more false positives.
- The test AUC of 0.8322 confirms strong discriminative ability between churners and non-churners on unseen data.

2. Classification Report (Test Set)

Class	Precision	Recall	F1-Score	Support
0 (Non-Churn)	0.8843	0.7903	0.8347	1035
1 (Churn)	0.5517	0.7139	0.6224	374
Accuracy			0.7700	1409
Macro Avg	0.7180	0.7521	0.7285	1409
Weighted Avg	0.7960	0.7700	0.7783	1409

The model performs better on the dominant Non-Churn class than on the Churn class. Non-Churn precision (0.8843) and F1 score (0.8347) are high, indicating accurate identification of loyal customers. The Churn class shows lower precision (0.5517) and a moderate F1 score (0.6224), reflecting that churn predictions are less precise. However, recall for the Churn class (0.7139) remains reasonably strong, meaning the model captures a large proportion of actual churners. The disparity between train and test performance is primarily due to class imbalance in the test set, since SMOTE was applied only during training. The weighted average F1 (0.7783) reflects overall balanced attention to both classes.

3. Confusion Matrix



- True Negatives (818): Correctly predicted non-churners
- True Positives (267): Correctly predicted churners
- False Positives (217): Predicted churn but actually non-churn
- False Negatives (107): Missed churners

The confusion matrix shows the model successfully identifies a strong proportion of actual churners (True Positives = 267) while keeping missed churners (False Negatives = 107) at a manageable level. The False Positives (217) are acceptable from a business standpoint, as it is far more costly to miss a churner than to incorrectly contact a loyal customer with a retention offer.

Summary

The XGBoost model demonstrates solid churn detection capability, achieving a test recall of 0.7139 and a test AUC of 0.8322. The use of SMOTE for class imbalance handling, combined with RandomizedSearchCV for hyperparameter optimization and threshold tuning at 0.40, produced a model calibrated for the business objective of minimizing missed churners. While mild overfitting is evident from the train-test performance gap which is characteristic of gradient boosting on SMOTE-augmented data the test performance is sufficiently robust for real-world churn prediction. The model reliably identifies the majority of churners while maintaining acceptable overall accuracy, making it a viable solution for early churn intervention.